

Actividad N°2 – Item II

Caso 1: NortíShop – Implementación de cifrado, hashing y tokenización

Seguridad y Protección de Datos – Proyecto Integrador

Integrantes: Ignacia Castillo, Milan Kurte, Francisco Polo

En este ítem se aplicó la protección definida en el cuadro del Ítem I a los 12 datos personales y financieros del caso NortíShop. Se hizo un programa en Python con una base de datos SQLite que guarda cada dato ya protegido, y se probó que el sistema puede validar o recuperar los datos cuando corresponde, y que las llaves no quedan expuestas.

Resumen de lo que se usó:

- **Hashing con bcrypt (costo 12)** para contraseñas del cliente y del administrador (solo se validan, nunca se recuperan).
- **Cifrado AES-256-GCM** para datos que sí se necesitan leer después: correos, RUT, teléfonos, direcciones, historial y comportamiento de compra. Cada valor queda amarrado a su fila (contexto AAD) y guarda la versión de la llave con que se cifró.
- **Tokenización** para las tarjetas: la tarjeta va a la pasarela de pago y NortíShop solo guarda un token y los últimos 4 dígitos.
- **Índice HMAC-SHA256** para poder buscar a un cliente por su correo sin tener que descifrar toda la tabla.
- **Firma HMAC-SHA256** de las respuestas de la pasarela de pago, para que no se pueda falsificar ni reutilizar un pago aprobado.

1. Código fuente

El código completo está en el repositorio público de GitHub: <https://github.com/m1lvn/nortishop-seguridad>. El proyecto tiene los siguientes archivos:

Archivo	Qué hace
config.py	Lee las llaves desde variables de entorno (.env). Si falta una llave, el programa no parte.
cripto.py	Hashing (bcrypt), cifrado/descifrado AES-256-GCM con contexto y versión de llave, índice HMAC, firma de pagos y enmascarado.
pasarela_simulada.py	Simula la pasarela de pago externa: recibe la tarjeta, devuelve un token y firma sus respuestas de cobro.
db.py	Estructura de la base de datos SQLite.
nortishop.py	Funciones del sistema: registro, login, compras, despacho, historial, boletín, etc.
generar_llaves.py	Genera las llaves aleatorias de 256 bits y crea el .env con permisos 600.
demo.py	Prueba de funcionamiento y pruebas de seguridad con datos ficticios.
ver_bd.py	Muestra la base de datos en crudo, como la vería alguien que la roba.
verificar_secretos.py	Revisa que las llaves no queden expuestas y que no haya datos personales en texto plano.
.gitignore / .env.example	Excluyen el .env y la BD del repositorio; el ejemplo no tiene llaves reales.
evidencia/	Salida real de cada script (ev_*.txt) y las capturas de las Figuras 1 a 6.
docs/	PDF de este informe (Ítem II) y del cuadro del Ítem I.

Para ejecutarlo:

```
pip install -r requirements.txt
python generar_llaves.py
python demo.py
python ver_bd.py
python verificar_secretos.py
```

El código completo de los archivos principales está en el Anexo al final del documento.

2. Protección de los campos críticos

La siguiente tabla muestra cada dato personal o financiero del Item I, la técnica que se le aplicó y dónde queda guardado en la base de datos. Los nombres de las columnas indican cómo está protegido el dato (_cifrado, _hash, _indice, token_).

Dato	Técnica aplicada	Dónde se implementa (tabla.columna / función)
Teléfono del cliente registrado	Cifrado AES-256	clientes.telefono_cifrado
Correo y contraseña de la cuenta	Cifrado AES-256 (correo) + Hashing bcrypt (contraseña)	clientes.correo_cifrado, clientes.correo_indice, clientes.password_hash
Tarjeta guardada para pagar más rápido	Tokenización	nortishop.guardar_tarjeta() → tarjetas_guardadas.token_pasarela
Dirección de despacho	Cifrado AES-256	clientes.direccion_cifrado, pedidos.direccion_cifrado
Historial de compras	Cifrado AES-256	pedidos.detalle_cifrado
RUT y datos para la boleta/factura	Cifrado AES-256	clientes.rut_cifrado
Correo y teléfono del invitado	Cifrado AES-256	pedidos.invitado_correo_cifrado, pedidos.invitado_telefono_cifrado
Datos de la tarjeta al pagar (invitado)	Tokenización	nortishop.comprar_como_invitado() → pedidos.token_pasarela
Clave del panel de administración	Hashing bcrypt	administradores.password_hash
Monto y tarjeta (tokenizada) de la compra	Tokenización	pedidos.monto (en claro) + pedidos.token_pasarela + pedidos.ultimos4; respuesta de la pasarela verificada con verificar_respuesta_pago()
Lista de correos del boletín	Cifrado AES-256	boletin_suscriptores.correo_cifrado (+ correo_indice)
Datos de comportamiento de compra	Cifrado AES-256	comportamiento_compra.datos_cifrado

Cómo funciona cada técnica en el código

Hashing (cripto.py → hashear_password): se usa bcrypt con un salt aleatorio y factor de costo 12. El salt hace que dos usuarios con la misma clave tengan hashes distintos, y el costo hace que probar claves por fuerza bruta sea lento.

Cifrado (cripto.py → cifrar): se usa AES-256-GCM con una llave de 32 bytes que se lee del .env. Cada vez que se cifra se genera un nonce aleatorio de 12 bytes, por eso el mismo dato cifrado dos veces da un resultado distinto. Además GCM agrega una etiqueta de autenticación, así que si alguien modifica el dato en la BD el sistema lo detecta.

Contexto y versión de llave (cripto.py → cifrar / descifrar): cada valor se cifra con un contexto (AAD) de la forma tabla.columna:identificador de la fila, por ejemplo pedidos.direccion:NS-9A7AE71410BC. El contexto no se guarda, pero si no coincide al descifrar, GCM rechaza el dato; así un valor cifrado no se puede copiar a otra fila o columna (por ejemplo, para desviar un despacho a otra dirección). Además, cada valor se guarda con el prefijo de la versión de la llave (v1:). Para rotar la llave se agrega NORTISHOP_LLAVE_CIFRADO_V2 al .env: los datos nuevos se cifran con v2 y los antiguos se siguen leyendo con v1.

Tokenización (pasarela_simulada.py → tokenizar): la tarjeta se manda a la pasarela de pago, que la guarda en su propia bóveda y devuelve un token del tipo tok_xxxx. Nortishop guarda solo ese token, los últimos 4 dígitos y la marca. El CVV no se guarda en ningún lado. Como no se puede conectar a

Transbank o Stripe real en un trabajo académico, la pasarela está simulada, pero funciona igual que una real.

Índice de búsqueda (cripto.py → indice_búsqueda): como el correo cifrado cambia cada vez, no se puede buscar con un WHERE. Por eso se guarda además un HMAC-SHA256 del correo (con otra llave distinta) que sirve solo para buscar y no permite recuperar el correo.

Firma de la pasarela (cripto.py → firmar_respuesta / firma_valida; nortishop.py → verificar_respuesta_pago): la pasarela firma cada respuesta de cobro con HMAC-SHA256, usando un secreto compartido con el comercio (NORTISHOP_LLAVE_PASARELA). Antes de registrar un pedido, Nortishop recalcula la firma y comprueba que la respuesta corresponda a ese pedido y a ese monto. Así no se puede falsificar un "aprobado" ni reutilizar la respuesta de otra compra, que era uno de los riesgos de integridad identificados en la Semana 1.

3. Evidencia de funcionamiento

3.1 Ejecución de la demo

Se generaron llaves nuevas y se corrió demo.py, que registra un cliente, un administrador, compras con tarjeta guardada y como invitado, suscriptores al boletín y datos de comportamiento. Todos los datos son ficticios.

```
~/UDD/Seguridad_Proteccion_Datos/nortishop_seguridad
python demo.py | tee evidencia/ev_demo.txt

=====
1. REGISTRO DE CLIENTE (correo, RUT, teléfono y dirección cifrados; contraseña hasheada)
=====
Cliente registrado con id 1

=====
2. VALIDACIÓN DE CONTRASEÑAS (hashing bcrypt: se compara, no se recupera)
=====
Login cliente con clave correcta -> ACEPTADO
Login cliente con clave incorrecta -> RECHAZADO
Login con correo en MAYÚSCULAS -> ACEPTADO (el índice HMAC normaliza el correo)
Login admin con clave correcta -> ACEPTADO
Login admin con clave incorrecta -> RECHAZADO

=====
3. RECUPERACIÓN DE DATOS CIFRADOS (AES-256-GCM, solo con la llave)
=====
nombre      : Camila Rojas
correo      : camila.rojas@correo.cl
rut         : 12.345.678-5
telefono    : +56 9 1234 5678
direccion   : Av. Los Leones 1234, Providencia
```

Figura 1: registro, validación de contraseñas y recuperación de datos cifrados

```
~/UDD/Seguridad_Proteccion_Datos/nortishop_seguridad

=====
4. TOKENIZACIÓN DE TARJETAS Y PAGO CON RESPUESTA FIRMADA
=====
Tarjeta guardada del cliente -> token: tok_4052b0710ffeab0bfa06a165
Compra con tarjeta guardada -> APROBADO, pedido NS-9A7AE71410BC, monto $51970, autorización CA98EEB6
Compra como invitado -> APROBADO, pedido NS-12ECDEC7D701, monto $14990, autorización 3E66CA58
Firma de la pasarela verificada antes de registrar cada pedido (HMAC-SHA256)

=====
5. HISTORIAL, DATOS DEL INVITADO Y DESPACHO (se descifran al consultar)
=====
Pedido NS-9A7AE71410BC - total $51970 - pagado con **** 1111
  1 x Audifonos Bluetooth ($39990)
  2 x Cable USB-C ($5990)
Contacto invitado pedido 2: {'correo': 'invitado.perez@mail.com', 'telefono': '+56 9 8765 4321'}
Dirección de despacho pedido 2: Calle Prat 456, Valparaíso

=====
6. MARKETING: BOLETÍN Y COMPORTAMIENTO DE COMPRA (cifrados)
=====
Correos descifrados solo al momento de enviar el boletín:
camila.rojas@correo.cl
juan.soto@gmail.com
maria.diaz@outlook.com
Comportamiento de compra descifrado:
categorias_favoritas: ['audio', 'accesorios']
compras_ultimo_anio: 7
ticket_promedio: 32500

=====
7. PRUEBAS DE SEGURIDAD
=====
Mismo correo cifrado dos veces da resultados distintos: True
v1:tu75YmLK1NfJKvQY2Najx080sSddKCGHsJt6QGk2LT0+ja2 ...
v1:hjIwRaxFT/RVUD9tw9ZNSRTsZzXyLSdzFc9CT2if/8HL/2W ...
El prefijo 'v1:' indica la versión de la llave usada (permite rotar llaves)
Dato alterado en la BD -> InvalidTag: el sistema rechaza el dato modificado
Llave incorrecta -> InvalidTag: sin la llave del .env no se recupera nada
Dirección copiada a otro pedido -> InvalidTag: el contexto no coincide, no se despacha a otra dirección
Pago 'rechazado' cambiado a 'aprobado' -> RECHAZADO (la firma ya no coincide)
Pago de $990 reutilizado en otro pedido -> RECHAZADO (no corresponde al pedido ni al monto)
Pago auténtico para su propio pedido -> ACEPTADO

Tarjeta mostrada en pantalla enmascarada: *****1111

Demo terminada.
👉
```

Figura 2: tokenización, historial, marketing y pruebas de seguridad

En la Figura 1 se ve que el login con la clave correcta es aceptado y con la clave incorrecta es rechazado, y que los datos cifrados (incluido el teléfono) se recuperan bien con la llave. En la Figura 2 se ve que las compras se aprueban usando solo el token y con la respuesta de la pasarela verificada, y en la parte 7 que el sistema rechaza un dato cifrado modificado, una llave incorrecta y una dirección copiada desde otro pedido (InvalidTag), además de una respuesta de pago falsificada o reutilizada en otro pedido (RECHAZADO).

3.2 Cómo quedan guardados los datos en la base de datos

Con `ver_bd.py` se muestra la base de datos en crudo, que es lo que vería alguien que se roba el archivo `nortishop.db` o un administrador de BD sin acceso a las llaves:

```
~/UDD/Seguridad_Proteccion_Datos/nortishop_seguridad  
python ver_bd.py | tee evidencia/ev_bd.txt
```

TABLA: clientes

id	= 1
nombre	= Camila Rojas
correo_cifrado	= v1:Eu2jTS0Zk28PguZwI6IL/qOm5xQb1fesVxUd/gSpbGnW...
correo_indice	= cc6082263c3a77dde17202919c0969e9f25c79a80efef20b...
password_hash	= \$2b\$12\$ikpuHs6pkXcY4/2IHAN0Yerrqdhkn7dIQofmUlb2v...
rut_cifrado	= v1:jAJH5LS/tWQ14VWbB3Npb2aaQQysMALP1TMcm/HZK9SxF...
telefono_cifrado	= v1:oPqcC9/cPdYVDYZvzT3nre73/ttAnMSajz4i9YLz/Yqe6...
direccion_cifrado	= v1:tBqC+WnwfnP6Hr1DdLEWeVZqMGaZRA876Behy1J/Y1Fi0...
nortipuntos	= 0

TABLA: tarjetas_guardadas

id	= 1
cliente_id	= 1
token_pasarela	= tok_4052b0710ffeab0bfa06a165
ultimos4	= 1111
marca	= VISA

TABLA: pedidos

id	= 1
codigo	= NS-9A7AE71410BC
cliente_id	= 1
es_invitado	= 0
invitado_correo_cifrado	= None
invitado_telefono_cifrado	= None
direccion_cifrado	= v1:oxR+tWfCXm4cTxTE+3rCVu4T0KSGTpq0Vi13yfEIV016o...
detalle_cifrado	= v1:XlPrczt3eAdPjau0Uwi5S0EYLn7avgIkuyb8C39FN481Y...
monto	= 51970
token_pasarela	= tok_4052b0710ffeab0bfa06a165
ultimos4	= 1111
autorizacion	= CA98EEB6
fecha	= 2026-10-02 01:01:15
id	= 2
codigo	= NS-12ECDEC7D701
cliente_id	= None
es_invitado	= 1
invitado_correo_cifrado	= v1:5QrzpnTDI50597He1pPHY/Ks0RjmxuTvPpGmnLxh/jhg...
invitado_telefono_cifrado	= v1:HyURGGQP1bVuUSyU6vDT24t+SXDafIOY555p8qBA0vxpf...
direccion_cifrado	= v1:Y00L5EL/iQ7nlanbqvpVgzQK0fCUPsj77Jt7AaooAZkd7...
detalle_cifrado	= v1:2fZ77GSGSgVud2D3YXFKidiW8rRn1WY9E5bvVUJcUMkxf...
monto	= 14990
token_pasarela	= tok_d1cc0cebe6d35c79a28a2861
ultimos4	= 2222
autorizacion	= 3E66CA58
fecha	= 2026-10-02 01:01:15

Figura 3: tablas clientes, tarjetas_guardadas y pedidos

TABLA: administradores	

id	= 1
usuario	= admin_norti
password_hash	= \$2b\$12\$V7jgiMRh3xDXMyQoFYU3D.dZm5/bV2hpTetSylWIq...

TABLA: boletin_suscriptores	

id	= 1
correo_cifrado	= v1:sFWmNw5gmoooklLqR3EIuZUgsZB1o2Fa/QLoPn5GF2i03...
correo_indice	= cc6082263c3a77dde17202919c0969e9f25c79a80efef20b...
id	= 2
correo_cifrado	= v1:2DzjvdD2d4npIiQM+ssvhCF/TPjCuu1StCtEwuyLIyaIG...
correo_indice	= e92e4c50071ba5f614ccc924304fe899dc1c35e905d21c0f...
id	= 3
correo_cifrado	= v1:dZuTgDGe9LiCHkQnxAEuP8a3ZazCtOgawRTNHatpOehwQ...
correo_indice	= 3f6841051ac903672c799258e55854a40ec1bc7845cdb847...

TABLA: comportamiento_compra	

id	= 1
cliente_id	= 1
datos_cifrado	= v1:M7vqGzItRy0cGzpJ/prg8s5XgTexsGr3/H14hC9pm2xbo...

Figura 4: tablas administradores, boletin_suscriptores y comportamiento_compra

Se puede ver que ningún dato personal o financiero aparece en texto plano: los correos, RUT, teléfonos, direcciones, historial y comportamiento aparecen como texto cifrado en base64 con el prefijo de versión v1:, las contraseñas como hash bcrypt (\$2b\$12\$...) y las tarjetas solo como token y últimos 4 dígitos. El nombre y el monto se dejaron en claro: el monto no identifica a la persona y se necesita para la contabilidad y la conciliación con la pasarela, y el nombre por sí solo (sin correo, RUT, teléfono ni dirección, que sí están cifrados) no permite contactar ni ubicar a la persona.

4. Explicación de cómo se recupera o valida el dato

4.1 Datos con hashing (contraseña del cliente y clave del administrador)

Estos datos no se recuperan nunca, solo se validan. El proceso de login es:

- El usuario escribe su correo y su contraseña.
- Se calcula el HMAC del correo y se busca la fila en la tabla con correo_indice.
- Se toma el password_hash guardado y se llama a bcrypt.checkpw(), que vuelve a hashear la clave escrita usando el mismo salt y compara los resultados.
- Si coinciden el login se acepta, si no se rechaza. En ningún momento se puede obtener la contraseña original desde el hash.

Si el cliente olvida la contraseña no se le puede enviar, hay que hacer un proceso de recuperación donde crea una nueva.

4.2 Datos cifrados con AES-256-GCM

Estos datos sí se recuperan, pero solo cuando se necesitan y solo con la llave del .env:

- Se lee el valor de la columna `_cifrado`, se separa el prefijo de versión (v1:) y el resto se decodifica de base64.
- Los primeros 12 bytes son el nonce y el resto es el texto cifrado con su etiqueta de autenticación.
- Con la versión se elige la llave y se llama a `AESGCM(llave).decrypt(nonce, cifrado, contexto)`, donde el contexto es `tabla.columna:identificador de la fila`. Primero se verifica la etiqueta y después se devuelve el texto original.
- Si el dato fue modificado, se copió desde otra fila o la llave es incorrecta, se lanza `InvalidTag` y no se devuelve nada.

Cuándo se descifra cada dato:

Dato	Cuándo se descifra	Función
Correo del cliente	Al enviar confirmaciones o al mostrar el perfil	<code>obtener_datos_cliente()</code>
RUT	Al generar la boleta o factura	<code>obtener_datos_cliente()</code>
Teléfono del cliente	Al coordinar la entrega o contactar al cliente	<code>obtener_datos_cliente()</code>
Dirección de despacho	Al crear la orden de despacho	<code>direccion_despacho()</code>
Historial de compras	Cuando el cliente revisa sus pedidos	<code>historial_compras()</code>
Correo y teléfono del invitado	Al notificar el estado del pedido	<code>datos_contacto_invitado()</code>
Lista del boletín	Solo al momento de enviar la campaña	<code>correos_para_envio_boletin()</code>
Comportamiento de compra	Cuando marketing arma la segmentación	<code>leer_comportamiento()</code>

4.3 Datos tokenizados (tarjetas)

NortiShop nunca recupera el número de la tarjeta, porque no lo tiene. Para cobrar se hace lo siguiente:

- Se busca el token guardado del cliente (o el del pedido del invitado).
- Se genera el código del pedido (NS-...) y se llama a `pasarela.cobrar(token, monto, pedido)`; la `pasarela`, que es la única que tiene la tarjeta real en su bóveda, hace el cobro.
- La `pasarela` responde aprobado/rechazado, un código de autorización y una firma HMAC. NortiShop verifica la firma, el pedido y el monto con `verificar_respuesta_pago()`, y solo entonces registra el pedido con su código de autorización.
- Al cliente se le muestra la tarjeta como `**** 1111` usando los últimos 4 dígitos.

Si alguien se roba la base de datos de NortiShop, los tokens no le sirven: solo funcionan con la `pasarela`, para este comercio y con la credencial del comercio, que no está en la base de datos.

5. Evidencia de que las llaves/secretos no quedan expuestos

Medidas que se tomaron para proteger las llaves:

- **No hay llaves en el código:** las llaves se generan con `os.urandom(32)` en `generar_llaves.py` y se guardan en `.env`. Ningún archivo `.py` tiene una llave escrita.
- **Variables de entorno:** `config.py` lee las llaves con `os.getenv()`. Si no existen, el programa se detiene con error en vez de usar una llave por defecto.
- **Permisos restringidos:** el `.env` tiene permisos 600 (`-rw-----`), solo el dueño lo puede leer.
- **Fuera del repositorio:** `.env` y `nortishop.db` están en `.gitignore`, así que no se suben al repositorio. Para que otro integrante sepa qué variables necesita existe `.env.example`, sin valores reales.
- **Llaves separadas:** se usan tres llaves distintas: una para cifrar (versionada, para poder rotarla), otra para el índice HMAC y otra para verificar las respuestas de la pasarela. Si se compromete una, las otras siguen protegidas.
- **Sin llaves en BD ni en logs:** la llave nunca se guarda en la base de datos ni se imprime en pantalla, y las tarjetas se muestran enmascaradas.

Para comprobarlo se hizo el script `verificar_secretos.py`, que revisa todo lo anterior automáticamente: busca el valor real de las tres llaves en el código, el README y los archivos de evidencia, y dentro de la base de datos, y además busca 14 datos personales y financieros de la demo para confirmar que no quedaron en texto plano:

```
~/UDD/Seguridad_Proteccion_Datos/nortishop_seguridad
➜ python verificar_secretos.py | tee evidencia/ev_secretos.txt
[OK] .env está incluido en .gitignore (no se sube al repositorio)
[OK] .env tiene permisos 600 (solo el dueño puede leerlo)
[OK] NORTISHOP_LLAVE_CIFRADO_V1 no aparece en el código, README ni evidencia (15 archivos revisados)
[OK] NORTISHOP_LLAVE_INDICE no aparece en el código, README ni evidencia (15 archivos revisados)
[OK] NORTISHOP_LLAVE_PASARELA no aparece en el código, README ni evidencia (15 archivos revisados)
[OK] NORTISHOP_LLAVE_CIFRADO_V1 no aparece dentro de la base de datos
[OK] NORTISHOP_LLAVE_INDICE no aparece dentro de la base de datos
[OK] NORTISHOP_LLAVE_PASARELA no aparece dentro de la base de datos
[OK] Ningún dato personal o financiero en texto plano en la BD (14 valores buscados)

Resultado: 9/9 verificaciones correctas
➜
```

Figura 5: resultado de `verificar_secretos.py` (9/9 verificaciones correctas)

También se comprobó con git que el `.env` y la base de datos quedan fuera del repositorio:

```

~/UDD/Seguridad_Proteccion_Datos/nortishop_seguridad
❶ cd ~/UDD/Seguridad_Proteccion_Datos/nortishop_seguridad
{ echo '$ ls -la'; command ls -la; echo
  echo '$ cat .env.example'; cat .env.example; echo
  echo '$ git ls-files # lo que está en el repositorio'; git ls-files; echo
  echo '$ git check-ignore -v .env nortishop.db'; git check-ignore -v .env nortishop.db; echo
  echo '$ grep -rLF "<llave real del .env>" *.py .env.example README.md evidencia/*.txt'
  command grep -rLF "${command grep LLAVE_CIFRADO_V1 .env | cut -d= -f2-}" *.py .env.example README.
    || echo "(sin resultados: la llave no aparece en el código, README ni evidencia)"
} | tee evidencia/ev_git.txt
$ ls -la
total 148
drwxr-xr-x 5 mllvn mllvn 4096 Oct 1 22:01 .
drwxrwxr-x 4 mllvn mllvn 4096 Oct 1 21:55 ..
-rw----- 1 mllvn mllvn 241 Oct 1 21:50 .env
-rw-r--r-- 1 mllvn mllvn 264 Oct 1 21:49 .env.example
drwxrwxr-x 7 mllvn mllvn 4096 Oct 1 22:10 .git
-rw-r--r-- 1 mllvn mllvn 71 Oct 1 21:48 .gitignore
-rw-r--r-- 1 mllvn mllvn 4709 Oct 1 21:54 README.md
drwxrwxr-x 2 mllvn mllvn 4096 Oct 1 21:54 __pycache__
-rw-r--r-- 1 mllvn mllvn 2070 Oct 1 21:49 config.py
-rw-r--r-- 1 mllvn mllvn 4099 Oct 1 21:49 cripto.py
-rw-r--r-- 1 mllvn mllvn 2529 Oct 1 21:54 db.py
-rw-r--r-- 1 mllvn mllvn 7320 Oct 1 21:54 demo.py
drwxr-xr-x 2 mllvn mllvn 4096 Oct 1 22:10 evidencia
-rw-r--r-- 1 mllvn mllvn 948 Oct 1 21:50 generar_llaves.py
-rw-r--r-- 1 mllvn mllvn 49152 Oct 1 22:01 nortishop.db
-rw-r--r-- 1 mllvn mllvn 9125 Oct 1 21:54 nortishop.py
-rw-r--r-- 1 mllvn mllvn 2084 Oct 1 21:49 pasarela_simulada.py
-rw-r--r-- 1 mllvn mllvn 50 Oct 1 20:29 requirements.txt
-rw-r--r-- 1 mllvn mllvn 738 Oct 1 21:50 ver_bd.py
-rw-r--r-- 1 mllvn mllvn 2474 Oct 1 21:54 verificar_secretos.py

$ cat .env.example
# Copia de ejemplo SIN llaves reales. Ejecutar generar_llaves.py para crear el .env verdadero.
NORTISHOP_LLAVE_CIFRADO_V1=<32 bytes en base64>
NORTISHOP_LLAVE_INDICE=<32 bytes en base64>
NORTISHOP_LLAVE_PASARELA=<32 bytes en base64>
NORTISHOP_RUTA_BD=nortishop.db

$ git ls-files # lo que está en el repositorio
.env.example
.gitignore
README.md
config.py
cripto.py
db.py
demo.py
evidencia/cap_bd_1.png
evidencia/cap_bd_2.png
evidencia/cap_demo_1.png
evidencia/cap_demo_2.png
evidencia/cap_secretos.png
evidencia/ev_bd.txt
evidencia/ev_demo.txt
evidencia/ev_git.txt
evidencia/ev_secretos.txt
generar_llaves.py
nortishop.py
pasarela_simulada.py
requirements.txt
ver_bd.py
verificar_secretos.py

$ git check-ignore -v .env nortishop.db
.gitignore:2:.env .env
.gitignore:3:*.db nortishop.db

$ grep -rLF "<llave real del .env>" *.py .env.example README.md evidencia/*.txt
(sin resultados: la llave no aparece en el código, README ni evidencia)
❷

```

Figura 6: permisos del .env, .env.example sin llaves, git ls-files, git check-ignore y búsqueda de la llave

En la Figura 6 se ve que el .env tiene permisos -rw-----, que git ls-files (la lista de archivos que están en el repositorio) no incluye el .env ni nortishop.db, y que git check-ignore confirma que los excluye la regla del .gitignore. La búsqueda del valor real de la llave en el código, el README y la evidencia no encuentra resultados. Lo mismo se puede comprobar en el repositorio público de GitHub.

Como mejora para un entorno real de producción, las llaves deberían estar en un gestor de secretos (por ejemplo AWS KMS, Azure Key Vault o HashiCorp Vault) y rotarse cada cierto tiempo, en vez de estar en un archivo .env en el mismo servidor.

Anexo: código fuente de los archivos principales

Código de los archivos principales tal como está en el repositorio (commit 77ac6c2). La versión completa y actualizada, incluidos demo.py y ver_bd.py, está en <https://github.com/m1lvn/nortishop-seguridad>.

config.py

```
# config.py
# Carga las llaves desde el archivo .env (variables de entorno).
# Las llaves NUNCA se escriben en el código, solo se leen desde el entorno.

import os
import re
import base64
from dotenv import load_dotenv

load_dotenv()

def _decodificar(nombre, valor):
    llave = base64.b64decode(valor)
    if len(llave) != 32:
        raise RuntimeError(f"{nombre} debe ser de 32 bytes (256 bits)")
    return llave

def _leer_llave(nombre):
    valor = os.getenv(nombre)
    if not valor:
        # si falta la llave el programa no parte, así no se usa una llave por defecto
        raise RuntimeError(f"Falta la variable de entorno {nombre}. Ejecuta generar_llaves.py primero.")
    return _decodificar(nombre, valor)

def _leer_llaves_cifrado():
    """
    Lee todas las versiones de la llave de cifrado (NORTISHOP_LLAVE_CIFRADO_V1, _V2, ...).
    Para rotar se agrega una versión nueva: los datos nuevos se cifran con la más reciente
    y los antiguos se siguen pudiendo leer con la versión que indica su prefijo (v1:, v2:, ...).
    """
    llaves = {}
    for nombre, valor in os.environ.items():
        m = re.fullmatch(r"NORTISHOP_LLAVE_CIFRADO_V(\d+)", nombre)
        if m and valor:
            llaves[f"v{m.group(1)}"] = _decodificar(nombre, valor)
    if not llaves:
        raise RuntimeError("Falta la variable de entorno NORTISHOP_LLAVE_CIFRADO_V1. Ejecuta generar_llaves.py primero.")
    return llaves

# llaves para cifrar con AES-256-GCM (RUT, dirección, correos, etc), por versión
LLAVES_CIFRADO = _leer_llaves_cifrado()

# versión con la que se cifran los datos nuevos (la más reciente)
VERSION_ACTUAL = max(LLAVES_CIFRADO, key=lambda v: int(v[1:]))

# llave aparte para el índice de búsqueda (HMAC) del correo
LLAVE_INDICE = _leer_llave("NORTISHOP_LLAVE_INDICE")

# secreto compartido con la pasarela para verificar la firma de sus respuestas
LLAVE_PASARELA = _leer_llave("NORTISHOP_LLAVE_PASARELA")

# factor de costo de bcrypt para las contraseñas
BCRYPT_COSTO = 12
```

```
RUTA_BD = os.getenv("NORTISHOP_RUTA_BD", "nortishop.db")
```

cripto.py

```
# cripto.py
# Funciones de protección de datos:
# - hashing de contraseñas (bcrypt)
# - cifrado / descifrado AES-256-GCM con contexto (AAD) y versión de llave
# - índice de búsqueda con HMAC-SHA256 (para buscar por correo sin descifrar todo)
# - firma HMAC-SHA256 de las respuestas de la pasarela

import os
import base64
import hmac
import hashlib
import json

import bcrypt
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

from config import LLAVES_CIFRADO, VERSION_ACTUAL, LLAVE_INDICE, BCRYPT_COSTO

# ----- HASHING (contraseñas) -----

def hashear_password(password):
    """Genera el hash bcrypt con salt aleatorio. No se puede revertir."""
    salt = bcrypt.gensalt(rounds=BCRYPT_COSTO)
    return bcrypt.hashpw(password.encode("utf-8"), salt).decode("utf-8")

def verificar_password(password_ingresada, hash_guardado):
    """Valida la contraseña comparando hashes (no se descifra nada)."""
    return bcrypt.checkpw(password_ingresada.encode("utf-8"), hash_guardado.encode("utf-8"))

# ----- CIFRADO (AES-256-GCM) -----
# Formato guardado en la BD: v1:<base64(nonce + texto cifrado + etiqueta)>
# - "v1" indica con qué versión de llave se cifró (permite rotar llaves)
# - el contexto (AAD) amarra el dato a su tabla, columna y fila: no se cifra,
#   pero si no coincide al descifrar, GCM rechaza el dato. Así un valor cifrado
#   no se puede copiar a otra fila o columna.

def cifrar(texto, contexto):
    """Cifra un texto con la llave actual. Devuelve 'version:base64'."""
    if texto is None:
        return None
    aes = AESGCM(LLAVES_CIFRADO[VERSION_ACTUAL])
    nonce = os.urandom(12) # nonce distinto en cada cifrado
    cifrado = aes.encrypt(nonce, texto.encode("utf-8"), contexto.encode("utf-8"))
    return VERSION_ACTUAL + ":" + base64.b64encode(nonce + cifrado).decode("utf-8")

def descifrar(texto_cifrado, contexto):
    """
    Recupera el texto original. Si el dato fue modificado, se movió a otra fila
    o la llave no corresponde, GCM lo detecta y lanza InvalidTag.
    """
    if texto_cifrado is None:
        return None
    version, b64 = texto_cifrado.split(":", 1)
    if version not in LLAVES_CIFRADO:
        raise ValueError(f"No existe la llave de cifrado {version}")
    datos = base64.b64decode(b64)
    nonce, cifrado = datos[:12], datos[12:]
    aes = AESGCM(LLAVES_CIFRADO[version])
    return aes.decrypt(nonce, cifrado, contexto.encode("utf-8")).decode("utf-8")
```

```

def cifrar_json(objeto, contexto):
    """Para datos con estructura (historial, comportamiento de compra)."""
    return cifrar(json.dumps(objeto, ensure_ascii=False), contexto)

def descifrar_json(texto_cifrado, contexto):
    return json.loads(descifrar(texto_cifrado, contexto))

# ----- ÍNDICE DE BÚSQUEDA -----

def indice_busqueda(valor):
    """
    Como el cifrado usa un nonce aleatorio, el mismo correo cifrado dos veces
    da resultados distintos y no se puede buscar con WHERE. Por eso se guarda
    además un HMAC del correo (con otra llave) que sirve solo para buscar.
    """
    normalizado = valor.strip().lower().encode("utf-8")
    return hmac.new(LLAVE_INDICE, normalizado, hashlib.sha256).hexdigest()

# ----- FIRMA (respuesta de la pasarela) -----

def _mensaje_pago(resp):
    return f"{resp['orden']}|{resp['aprobado']}|{resp['codigo']}|{resp['autorizacion']}|{resp['monto']}"

def firmar_respuesta(llave, resp):
    """HMAC-SHA256 de los campos de la respuesta. Lo calcula la pasarela."""
    return hmac.new(llave, _mensaje_pago(resp).encode("utf-8"), hashlib.sha256).hexdigest()

def firma_valida(llave, resp):
    """Recalcula la firma y la compara en tiempo constante. Lo hace Nortishop."""
    esperada = firmar_respuesta(llave, resp)
    return hmac.compare_digest(esperada, resp.get("firma", ""))

def enmascarar(valor, visibles=4):
    """Para mostrar en pantalla/logs sin exponer el dato completo."""
    if not valor:
        return valor
    return "*" * max(len(valor) - visibles, 0) + valor[-visibles:]

```

pasarela_simulada.py

```

# pasarela_simulada.py
# Simula la pasarela de pago externa (tipo Transbank / Stripe).
# En la vida real esto es un servicio de un tercero: Nortishop le manda la
# tarjeta UNA vez y recibe de vuelta un token. El número real queda guardado
# solo en la bóveda de la pasarela, nunca en la base de datos de Nortishop.
# Cada respuesta de cobro va firmada con un secreto compartido con el comercio,
# para que Nortishop pueda comprobar que la respuesta es auténtica.

import secrets

from cripto import firmar_respuesta

class PasarelaPago:
    def __init__(self, llave_firma):
        # "bóveda" de la pasarela: vive fuera de Nortishop (aquí, solo en memoria)
        self._bodega = {}

```

```

        self._llave_firma = llave_firma

    def tokenizar(self, numero_tarjeta, vencimiento, cvv):
        """Recibe la tarjeta y devuelve un token + los últimos 4 dígitos."""
        numero = numero_tarjeta.replace(" ", "")
        if not numero.isdigit() or not (13 <= len(numero) <= 19):
            raise ValueError("Número de tarjeta inválido")
        token = "tok_" + secrets.token_hex(12)
        # el CVV no se guarda nunca, ni siquiera en la pasarela (norma PCI)
        self._boveda[token] = {"numero": numero, "vencimiento": vencimiento}
        return {"token": token, "ultimos4": numero[-4:], "marca": self._marca(numero)}

    def cobrar(self, token, monto, orden):
        """Cobra usando el token. NortiShop nunca necesita el número real."""
        if token not in self._boveda:
            resp = {"orden": orden, "aprobado": False, "codigo": "TOKEN_INVALIDO",
"autorizacion": "", "monto": monto}
        else:
            resp = {"orden": orden, "aprobado": True, "codigo": "APROBADO",
"autorizacion": secrets.token_hex(4).upper(), "monto": monto}
            resp["firma"] = firmar_respuesta(self._llave_firma, resp)
        return resp

    @staticmethod
    def _marca(numero):
        if numero.startswith("4"):
            return "VISA"
        if numero[:2] in ("51", "52", "53", "54", "55"):
            return "MASTERCARD"
        return "OTRA"

```

db.py

```

# db.py
# Estructura de la base de datos (SQLite).
# Los nombres de columna indican cómo está protegido cada campo:
# * _cifrado -> AES-256-GCM (con contexto tabla.columna:identificador de la fila)
# * _hash -> bcrypt
# * _indice -> HMAC (solo para buscar)
# token_* -> token de la pasarela

import sqlite3
from config import RUTA_BD

ESQUEMA = """
CREATE TABLE IF NOT EXISTS clientes (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre            TEXT NOT NULL,
    correo_cifrado    TEXT NOT NULL,
    correo_indice     TEXT NOT NULL UNIQUE,
    password_hash     TEXT NOT NULL,
    rut_cifrado       TEXT,
    telefono_cifrado  TEXT,
    direccion_cifrado TEXT,
    nortipuntos       INTEGER DEFAULT 0    -- no es dato personal: su riesgo es de integridad
);

CREATE TABLE IF NOT EXISTS tarjetas_guardadas (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    cliente_id        INTEGER NOT NULL REFERENCES clientes(id),
    token_pasarela    TEXT NOT NULL,
    ultimos4          TEXT NOT NULL,
    marca             TEXT
);

CREATE TABLE IF NOT EXISTS pedidos (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    codigo            TEXT NOT NULL UNIQUE,
    cliente_id        INTEGER REFERENCES clientes(id),
    es_invitado       INTEGER DEFAULT 0,

```

```

    invitado_correo_cifrado TEXT,
    invitado_telefono_cifrado TEXT,
    direccion_cifrado TEXT NOT NULL,
    detalle_cifrado TEXT NOT NULL,
    monto INTEGER NOT NULL,
    token_pasarela TEXT NOT NULL,
    ultimos4 TEXT NOT NULL,
    autorizacion TEXT,
    fecha TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS administradores (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    usuario TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS boletin_suscriptores (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    correo_cifrado TEXT NOT NULL,
    correo_indice TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS comportamiento_compra (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cliente_id INTEGER NOT NULL REFERENCES clientes(id),
    datos_cifrado TEXT NOT NULL
);
"""

def conectar():
    con = sqlite3.connect(RUTA_BD)
    con.row_factory = sqlite3.Row
    return con

def crear_tablas():
    con = conectar()
    con.executescript(ESQUEMA)
    con.commit()
    con.close()

```

nortishop.py

```

# nortishop.py
# Funciones del sistema que guardan y leen los datos personales y financieros ya protegidos.
#
# Cada valor cifrado lleva un contexto "tabla.columna:identificador" que lo amarra
# a su fila. El identificador es uno que no cambia y se conoce antes del INSERT:
#   clientes / boletín -> correo_indice      pedidos -> codigo del pedido
#   comportamiento     -> cliente_id

import secrets

from db import conectar
from config import LLAVE_PASARELA
from cripto import (hashear_password, verificar_password, cifrar, descifrar,
                    cifrar_json, descifrar_json, indice_busqueda, firma_valida)

def _ctx_cliente(columna, correo_indice):
    return f"clientes.{columna}:{correo_indice}"

def _ctx_pedido(columna, codigo):
    return f"pedidos.{columna}:{codigo}"

```

```

# ----- Cliente registrado -----

def registrar_cliente(nombre, correo, password, rut, telefono, direccion):
    idx = indice_busqueda(correo)
    con = conectar()
    cur = con.execute(
        """INSERT INTO clientes (nombre, correo_cifrado, correo_indice, password_hash,
                                rut_cifrado, telefono_cifrado, direccion_cifrado)
        VALUES (?, ?, ?, ?, ?, ?, ?)""",
        (nombre, cifrar(correo, _ctx_cliente("correo", idx)), idx, hashear_password(password),
        cifrar(rut, _ctx_cliente("rut", idx)), cifrar(telefono, _ctx_cliente("telefono", idx)),
        cifrar(direccion, _ctx_cliente("direccion", idx))),
    )
    con.commit()
    con.close()
    return cur.lastrowid

def login_cliente(correo, password):
    """Valida el login: busca por el índice HMAC y compara el hash bcrypt."""
    con = conectar()
    fila = con.execute("SELECT id, password_hash FROM clientes WHERE correo_indice = ?",
        (indice_busqueda(correo),)).fetchone()
    con.close()
    if fila is None:
        return None
    return fila["id"] if verificar_password(password, fila["password_hash"]) else None

def obtener_datos_cliente(cliente_id):
    """Recupera (descifra) los datos del cliente solo cuando se necesitan."""
    con = conectar()
    f = con.execute("SELECT * FROM clientes WHERE id = ?", (cliente_id,)).fetchone()
    con.close()
    idx = f["correo_indice"]
    return {
        "nombre": f["nombre"],
        "correo": descifrar(f["correo_cifrado"], _ctx_cliente("correo", idx)),
        "rut": descifrar(f["rut_cifrado"], _ctx_cliente("rut", idx)),
        "telefono": descifrar(f["telefono_cifrado"], _ctx_cliente("telefono", idx)),
        "direccion": descifrar(f["direccion_cifrado"], _ctx_cliente("direccion", idx)),
    }

def guardar_tarjeta(cliente_id, pasarela, numero, vencimiento, cvv):
    """La tarjeta se manda a la pasarela y solo se guarda el token."""
    resp = pasarela.tokenizar(numero, vencimiento, cvv)
    con = conectar()
    con.execute("INSERT INTO tarjetas_guardadas (cliente_id, token_pasarela, ultimos4, marca)
VALUES (?, ?, ?, ?)",
        (cliente_id, resp["token"], resp["ultimos4"], resp["marca"]))
    con.commit()
    con.close()
    return resp["token"]

# ----- Compras -----

def verificar_respuesta_pago(pago, monto, orden):
    """
    Antes de registrar un pedido se comprueba que la respuesta venga de la pasarela
    (firma HMAC válida) y que corresponda a ESTE pedido y a ESTE monto. Así no se
    puede falsificar un "aprobado" ni reutilizar la respuesta de otra compra.
    """
    return (firma_valida(LLAVE_PASARELA, pago)
        and pago["orden"] == orden
        and pago["monto"] == monto)

```

```

def _nuevo_codigo_pedido():
    return "NS-" + secrets.token_hex(6).upper()

def _pagar(pasarela, token, monto, orden):
    pago = passarela.cobrar(token, monto, orden)
    if not verificar_respuesta_pago(pago, monto, orden):
        return {"aprobado": False, "codigo": "RESPUESTA_NO_VALIDA", "orden": orden}
    return pago

def _registrar_pedido(con, codigo, cliente_id, es_invitado, correo_inv, tel_inv, direccion,
productos, monto,
                    token, ultimos4, autorizacion):
    con.execute(
        """INSERT INTO pedidos (codigo, cliente_id, es_invitado, invitado_correo_cifrado,
invitado_telefono_cifrado,
                    direccion_cifrado, detalle_cifrado, monto, token_pasarela,
ultimos4, autorizacion)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (codigo, cliente_id, es_invitado,
        cifrar(correo_inv, _ctx_pedido("invitado_correo", codigo)),
        cifrar(tel_inv, _ctx_pedido("invitado_telefono", codigo)),
        cifrar(direccion, _ctx_pedido("direccion", codigo)),
        cifrar_json(productos, _ctx_pedido("detalle", codigo)),
        monto, token, ultimos4, autorizacion),
    )

def comprar_con_tarjeta_guardada(cliente_id, passarela, productos, monto):
    con = conectar()
    t = con.execute("SELECT token_pasarela, ultimos4 FROM tarjetas_guardadas WHERE cliente_id
= ?",
                    (cliente_id,)).fetchone()
    c = con.execute("SELECT direccion_cifrado, correo_indice FROM clientes WHERE id=?",
(cliente_id,)).fetchone()
    direccion = descifrar(c["direccion_cifrado"], _ctx_cliente("direccion", c["correo_indice"]))
    codigo = _nuevo_codigo_pedido()
    pago = _pagar(pasarela, t["token_pasarela"], monto, codigo)
    if pago["aprobado"]:
        _registrar_pedido(con, codigo, cliente_id, 0, None, None, direccion, productos, monto,
t["token_pasarela"], t["ultimos4"], pago["autorizacion"])
        con.commit()
    con.close()
    return pago

def comprar_como_invitado(pasarela, correo, telefono, direccion, numero, vencimiento, cvv,
productos, monto):
    """El invitado no tiene cuenta: su tarjeta se tokeniza para esta compra y sus datos se
cifran."""
    tok = passarela.tokenizar(numero, vencimiento, cvv)
    codigo = _nuevo_codigo_pedido()
    pago = _pagar(pasarela, tok["token"], monto, codigo)
    if pago["aprobado"]:
        con = conectar()
        _registrar_pedido(con, codigo, None, 1, correo, telefono, direccion, productos, monto,
tok["token"], tok["ultimos4"], pago["autorizacion"])
        con.commit()
        con.close()
    return pago

def historial_compras(cliente_id):
    con = conectar()

```

```

        filas = con.execute("SELECT codigo, detalle_cifrado, monto, ultimos4 FROM pedidos WHERE
cliente_id=?",
                            (cliente_id,)).fetchall()
        con.close()
        return [{"pedido": f["codigo"], "productos": descifrar_json(f["detalle_cifrado"],
_ctx_pedido("detalle", f["codigo"])),
                "monto": f["monto"], "tarjeta": "**** " + f["ultimos4"]} for f in filas]

def datos_contacto_invitado(pedido_id):
    con = conectar()
    f = con.execute("SELECT codigo, invitado_correo_cifrado, invitado_telefono_cifrado FROM
pedidos WHERE id=?",
                    (pedido_id,)).fetchone()
    con.close()
    return {"correo": descifrar(f["invitado_correo_cifrado"], _ctx_pedido("invitado_correo",
f["codigo"])),
            "telefono": descifrar(f["invitado_telefono_cifrado"],
_ctx_pedido("invitado_telefono", f["codigo"]))}

def direccion_despacho(pedido_id):
    """Se descifra la dirección solo al generar la orden de despacho."""
    con = conectar()
    f = con.execute("SELECT codigo, direccion_cifrado FROM pedidos WHERE id=?",
                    (pedido_id,)).fetchone()
    con.close()
    return descifrar(f["direccion_cifrado"], _ctx_pedido("direccion", f["codigo"]))

# ----- Administrador -----

def crear_admin(usuario, password):
    con = conectar()
    con.execute("INSERT INTO administradores (usuario, password_hash) VALUES (?,?)",
                (usuario, hashear_password(password)))
    con.commit()
    con.close()

def login_admin(usuario, password):
    con = conectar()
    f = con.execute("SELECT password_hash FROM administradores WHERE usuario=?",
                    (usuario,)).fetchone()
    con.close()
    return f is not None and verificar_password(password, f[0])

# ----- Marketing -----

def suscribir_boletin(correo):
    idx = indice_busqueda(correo)
    con = conectar()
    con.execute("INSERT OR IGNORE INTO boletin_suscriptores (correo_cifrado, correo_indice)
VALUES (?,?)",
                (cifrar(correo, f"boletin.correo:{idx}"), idx))
    con.commit()
    con.close()

def correos_para_envio_boletin():
    """Se descifra la lista solo al momento de enviar la campaña."""
    con = conectar()
    filas = con.execute("SELECT correo_cifrado, correo_indice FROM
boletin_suscriptores").fetchall()
    con.close()
    return [descifrar(f["correo_cifrado"], f"boletin.correo:{f['correo_indice']}") for f in
filas]

```

```
def guardar_comportamiento(cliente_id, datos):
    con = conectar()
    con.execute("INSERT INTO comportamiento_compra (cliente_id, datos_cifrado) VALUES (?,?)",
                (cliente_id, cifrar_json(datos, f"comportamiento.datos:{cliente_id}")))
    con.commit()
    con.close()

def leer_comportamiento(cliente_id):
    con = conectar()
    f = con.execute("SELECT datos_cifrado FROM comportamiento_compra WHERE cliente_id=?",
                    (cliente_id,)).fetchone()
    con.close()
    return descifrar_json(f["datos_cifrado"], f"comportamiento.datos:{cliente_id}")
```

generar_llaves.py

```
# generar_llaves.py
# Genera llaves aleatorias de 256 bits y las guarda en .env
# (el .env está en .gitignore, así que nunca se sube al repositorio)

import os
import base64
import stat

RUTA = ".env"

def nueva_llave():
    return base64.b64encode(os.urandom(32)).decode()

if os.path.exists(RUTA):
    print(".env ya existe, no se sobrescribe (si se pierde la llave no se pueden descifrar los datos).")
else:
    with open(RUTA, "w") as f:
        # versión 1 de la llave de cifrado; para rotar se agrega NORTISHOP_LLAVE_CIFRADO_V2
        f.write(f"NORTISHOP_LLAVE_CIFRADO_V1={nueva_llave()}\n")
        f.write(f"NORTISHOP_LLAVE_INDICE={nueva_llave()}\n")
        f.write(f"NORTISHOP_LLAVE_PASARELA={nueva_llave()}\n")
        f.write(f"NORTISHOP_RUTA_BD=nortishop.db\n")
    # permisos 600: solo el dueño del archivo lo puede leer
    os.chmod(RUTA, stat.S_IRUSR | stat.S_IWUSR)
    print(".env creado con llaves nuevas (permisos 600).")
```

verificar_secretos.py

```
# verificar_secretos.py
# Revisa que las llaves no queden expuestas:
# 1. .env está en .gitignore
# 2. .env tiene permisos 600 (solo lo lee el dueño)
# 3. ninguna llave aparece escrita en el código, el README ni la evidencia
# 4. ninguna llave aparece dentro de la base de datos
# 5. ningún dato personal o financiero quedó en texto plano en la base de datos

import os
import stat
import glob
import base64
from dotenv import dotenv_values

resultados = []

def check(descripcion, ok):
```

```

    resultados.append(ok)
    print(f"[{'OK' if ok else 'FALLA'}] {descripcion}")

def leer_bytes(ruta):
    with open(ruta, "rb") as f:
        return f.read()

env = dotenv_values(".env")
llaves = {k: v for k, v in env.items() if "LLAVE" in k}
ruta_bd = env.get("NORTISHOP_RUTA_BD", "nortishop.db")

# 1
with open(".gitignore") as f:
    check(".env está incluido en .gitignore (no se sube al repositorio)", ".env" in
f.read().split())

# 2
modo = stat.S_IMODE(os.stat(".env").st_mode)
check(f".env tiene permisos {oct(modo)[2:]} (solo el dueño puede leerlo)", modo == 0o600)

# 3
archivos_codigo = glob.glob("*.py") + [".env.example", "README.md"] +
glob.glob("evidencia/*.txt")
for nombre, valor in llaves.items():
    crudo = base64.b64decode(valor)
    encontrado = []
    for a in archivos_codigo:
        contenido = leer_bytes(a)
        if valor.encode() in contenido or crudo in contenido or crudo.hex().encode() in
contenido:
            encontrado.append(a)
        check(f"{nombre} no aparece en el código, README ni evidencia ({len(archivos_codigo)}
archivos revisados)",
            not encontrado)

# 4
bd = leer_bytes(ruta_bd)
for nombre, valor in llaves.items():
    crudo = base64.b64decode(valor)
    check(f"{nombre} no aparece dentro de la base de datos", valor.encode() not in bd and crudo
not in bd)

# 5
datos_personales = ["4111111111111111", "5555444433332222", "12.345.678-5",
"camila.rojas@correo.cl",
"Los Leones", "MiClave#2026", "Adm!nPanel2026", "invitado.perez@mail.com",
"8765 4321", "1234 5678", "Calle Prat", "Audífonos", "juan.soto@gmail.com",
"ticket_promedio"]
en_claro = [d for d in datos_personales if d.encode() in bd]
check(f"Ningún dato personal o financiero en texto plano en la BD ({len(datos_personales)}
valores buscados)",
    not en_claro)
if en_claro:
    print("    encontrados:", en_claro)

print(f"\nResultado: {sum(resultados)}/{len(resultados)} verificaciones correctas")

```